A ZOO Deep Learning Survey

Francois Chaubard
Stanford University
fchaubar@stanford.edu

Abstract

Zero-order optimization (ZOO) trains a network from loss evaluations alone, without back-propagating errors. It is the natural training rule for neuromorphic and analog hardware, for systems that contain non-differentiable components, for biologically plausible learning, and for memory-constrained fine-tuning. Many ZOO rules have been proposed, but they are rarely compared under one budget on the same networks. We survey eight training rules—backpropagation as the reference, simultaneous perturbation (SPSA), random search, four node-perturbation variants (classic, activity-based, iterative and decorrelated), and the forward-forward algorithm—on four standard ≈40k-parameter architectures: an MLP on MNIST, a convolutional network on CIFAR-10, and a character-level LSTM and Transformer on Penn Treebank. Every method is given the same 16-forward-pass step budget and the same one hour of wall-clock training on a single GPU, feeds its gradient estimate to Adam, and is swept over three probe scales, yielding 96 runs. We report final test loss, accuracy and bits-per-character, training curves against wall-clock, sensitivity to the probe scale, and throughput. Iterative node perturbation (INP) is the most consistent zero-order rule: it is the best probe method on both sequence tasks and within a point of the best on both image tasks, where activity-based (ANP) and decorrelated (DANP) node perturbation win. At their best evaluation the node-perturbation rules reach 96.7% on MNIST and 62.9% on CIFAR-10 against 97.5% and 69.7% for backpropagation, but on the character-level LSTM and Transformer the best zero-order rule still trails backpropagation by about one bit per character (2.82 versus 1.78 and 3.10 versus 1.82 bits/char). Weight-space perturbation (SPSA, random search) is competitive only on the sequence models, where the number of perturbed units approaches the number of weights, and the forward-forward readout lags far behind its own label-accumulation inference (19% versus 96% on MNIST). Code, configurations and raw logs are released at https://github.com/Fchaubard/zero-order-ablations.

1 Introduction

Backpropagation [Rumelhart et al., 1986] is the default way to train neural networks, but it is not always available. Neuromorphic and analog accelerators implement a forward pass natively and may have no mechanism to transport exact error signals backwards through the substrate; systems with non-differentiable components (simulators, hardware in the loop, discrete decisions, black-box APIs) expose only a loss value; neuroscience asks which learning rules a brain could plausibly implement with local information; and fine-tuning of very large models is increasingly limited by the activation memory that backpropagation must store [Malladi et al., 2023]. In all of these settings the learner can evaluate the loss at perturbed parameters or perturbed activities, but cannot differentiate it. *Zero-order optimization* (ZOO) is the family of methods that estimate a descent direction from such evaluations alone.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

The ZOO literature is old and broad: random search [Rastrigin, 1963, Matyas, 1965, Schumer and Steiglitz, 1968], simultaneous perturbation stochastic approximation [Spall, 1992, 1998], Gaussian smoothing and its convergence theory [Nesterov and Spokoiny, 2017, Duchi et al., 2015, Flaxman et al., 2005], evolution strategies [Wierstra et al., 2014, Hansen et al., 2003, Salimans et al., 2017, Mania et al., 2018], node perturbation [Widrow and Lehr, 1990, Werfel et al., 2003, Fiete and Seung, 2006, Hiratani et al., 2022] and its recent activity-based and decorrelated variants [Dalm et al., 2023], and local learning rules such as forward-forward [Hinton, 2022]. Each paper demonstrates its method on its own architecture, with its own budget of forward passes, its own choice of optimizer and its own hyper-parameter search. A practitioner who must train without backpropagation is left with the question this paper tries to answer: *which ZOO method wins, on standard small architectures, when every method is given exactly the same budget?*

We answer it empirically. Our protocol fixes four ≈40k-parameter models on four standard tasks (an MLP on MNIST, a ConvNet on CIFAR-10, and a character-level LSTM and Transformer on Penn Treebank), gives every probe-based method 16 forward passes per step, runs every method for one hour of wall-clock training on one GPU, feeds every method's gradient estimate to the same Adam optimizer, and sweeps a single scalar ε that is simultaneously the learning rate and the probe scale. Model selection uses validation loss only; test metrics are reported once.

Contributions.

- A single, fully specified protocol (Section 3 and Section 4) under which eight training rules—backpropagation, SPSA, random search, classic / activity-based / iterative / decorrelated node perturbation and forward-forward—are compared on four architectures with identical forward-pass, wall-clock and hyper-parameter budgets.
- A faithful implementation of each published estimator on top of one *perturbable layer* abstraction (Appendix B) so that every method drives the same models, including an LSTM and a pre-LN Transformer, not only MLPs.
- The resulting 96-run grid (Table 3, Appendix A), training curves against wall-clock (Figure 1), the sensitivity of every method to the probe scale (Figure 2) and the throughput of every method (Table 5), with code and logs released.

Table 3 previews the main result: iterative node perturbation is the most consistent zero-order rule (best on both sequence tasks, mean rank 2.0 of 7 across the four tasks), activity-based and decorrelated node perturbation win the two image tasks, and the best zero-order rule matches backpropagation's accuracy on MNIST to within a point but trails it by about one bit per character on the character-level LSTM and Transformer after one hour.

2 Related Work

SPSA, Gaussian smoothing and evolution strategies. Simultaneous perturbation stochastic approximation [Spall, 1992, 1998] estimates a gradient from two loss evaluations at $\boldsymbol{\theta} \pm \varepsilon \mathbf{z}$ with a random direction $\mathbf{z}$; with Gaussian $\mathbf{z}$ the estimator is the gradient of the Gaussian-smoothed loss [Nesterov and Spokoiny, 2017], and its complexity in the convex [Duchi et al., 2015] and bandit [Flaxman et al., 2005] settings is well understood (see Liu et al. [2020] for a survey). Evolution strategies use the same estimator with many directions, fitness shaping and natural-gradient corrections [Wierstra et al., 2014, Hansen et al., 2003, Hansen, 2016]; Salimans et al. [2017] and Mania et al. [2018] showed that they are competitive with policy-gradient reinforcement learning, which is itself a zero-order method with respect to the environment [Williams, 1992]. Adaptive-moment updates on top of zero-order estimates were studied by Chen et al. [2019]. Our SPSA and RS baselines are the two-point and best-of-K instances of this family.

Node perturbation. Rather than perturbing weights, node perturbation injects noise into the units of the network and correlates the resulting change of the loss with the noise, which yields a weight update that is local to each layer [Widrow and Lehr, 1990, Werfel et al., 2003, 2005, Fiete and Seung, 2006]. Hiratani et al. [2022] analysed its stability and scalability and showed that it suffers from weight diffusion in mismatched networks. Dalm et al. [2023] reformulated node perturbation as a directional derivative (*iterative* NP), proposed the cheaper *activity-based* NP that uses the measured

activity change rather than the injected noise, and combined it with the input decorrelation rule of Ahmad et al. [2023], which was also shown to accelerate backpropagation itself [Dalm et al., 2024, Ahmad, 2024]. We implement all four variants.

Forward gradients. Forward-mode automatic differentiation gives an unbiased directional derivative in a single forward pass; Baydin et al. [2022] proposed training with a random direction, and Ren et al. [2023] scaled it with local losses. Forward gradients require a Jacobian-vector product through the network and therefore do not apply to the black-box and hardware settings that motivate this survey; we do not include them.

Decorrelation. Correlated layer inputs make the loss landscape ill-conditioned; decorrelating inputs with a local rule [Ahmad et al., 2023] removes this and was reported to make node perturbation competitive with backpropagation [Dalm et al., 2023] and to speed up backpropagation [Dalm et al., 2024, Ahmad, 2024]. Our DANP baseline uses exactly this rule.

Forward-forward and other local rules. The forward-forward algorithm [Hinton, 2022] replaces the backward pass by two forward passes with positive and negative data and trains each layer to separate them with a local "goodness" objective. Other backprop-free rules, such as feedback alignment [Lillicrap et al., 2016], direct feedback alignment [Nøkland, 2016] and synthetic gradients [Jaderberg et al., 2017], still require a backward signal that is learned or random; they are outside our zero-order scope and we include only forward-forward as the representative local rule.

Zero-order fine-tuning of large language models. MeZO [Malladi et al., 2023] showed that two-point SPSA with in-place perturbation can fine-tune billion-parameter language models with inference-level memory; DeepZero [Chen et al., 2024] scaled zero-order training from scratch with sparsity and coordinate-wise estimates; and Zhang et al. [2024] benchmarked the resulting estimators and optimizers for memory-efficient fine-tuning. Those works vary the estimator on one large model; we vary the estimator *and* the architecture at a scale where every method can be run to its one-hour limit.

3 Methods

3.1 Problem setup and the shared update rule

Let $\boldsymbol{\theta} \in \mathbb{R}^P$ be the parameters of a network with L perturbable layers, and let $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} \ell_n(\boldsymbol{\theta})$ be the mini-batch cross-entropy loss, where ℓ_n is the per-example loss (for language modelling, the per-sequence mean of the 64 next-character cross-entropies). Every method in this paper produces an estimate $\hat{\mathbf{g}} \approx \nabla_{\boldsymbol{\theta}}\mathcal{L}$ of the gradient and writes it into the gradient slot of every parameter; backpropagation writes the exact gradient, the zero-order methods write their estimator, and forward-forward writes the gradient of its local objectives. The estimate is then consumed by the same optimizer, Adam [Kingma and Ba, 2015] with $(\beta_1, \beta_2) = (0.9, 0.99)$ and no schedule:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1-\beta_1)\,\hat{\mathbf{g}}, \qquad \mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1-\beta_2)\,\hat{\mathbf{g}}^2, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \varepsilon\,\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + 10^{-8}}, \tag{1}$$

with the usual bias corrections $\hat{\mathbf{m}}, \hat{\mathbf{v}}$. The learning rate ε is the only hyper-parameter, and it *doubles as the probe scale*: it is the standard deviation of the parameter perturbation for SPSA and RS and the standard deviation σ of the node noise for the node-perturbation family. All probes are drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and scaled by ε. Tying the two removes a two-dimensional grid search per method and expresses the natural constraint that a probe cannot be finer than the step it informs.

Because Adam normalises $\hat{\mathbf{g}}$ by a running estimate of its own magnitude, Eq. (1) is approximately invariant to a constant rescaling of the estimator. The normalisation constants of the estimators below therefore influence the first few hundred steps and the relative weighting of estimates across steps, but not the asymptotic step size; this is deliberate and makes the comparison insensitive to the different conventions in the literature.

3.2 Optimizers

SPSA (central difference, Gaussian probes). With $K = 8$ antithetic pairs of probes $\mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_P)$ the estimator is the two-point central difference

$$\hat{\mathbf{g}}_{\text{SPSA}} = \frac{1}{K} \sum_{k=1}^{K} \frac{\mathcal{L}(\boldsymbol{\theta} + \varepsilon \mathbf{z}_k) - \mathcal{L}(\boldsymbol{\theta} - \varepsilon \mathbf{z}_k)}{2\varepsilon} \mathbf{z}_k, \qquad K = 8, \tag{2}$$

which is the Gaussian-smoothing estimator of Nesterov and Spokoiny [2017] and the multi-direction form of SPSA [Spall, 1992] with Gaussian rather than Rademacher directions. Each pair costs two forward passes on the batch-mean loss, so the 16-pass budget buys $K = 8$ directions. The implementation mirrors the in-place perturbation of MeZO [Malladi et al., 2023]: the parameters are restored exactly from a flat copy after each pair. No clean pass is spent; the training loss reported for SPSA is the mean of the 16 probe losses.

Random search (jumping). Random search [Rastrigin, 1963, Matyas, 1965, Schumer and Steiglitz, 1968] evaluates the clean loss $\mathcal{L}_0 = \mathcal{L}(\boldsymbol{\theta})$ and $K = 15$ candidates $\boldsymbol{\theta} + \varepsilon \mathbf{z}_k$, $\mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_P)$, and keeps the best one, $k^* = \arg\min_k \mathcal{L}(\boldsymbol{\theta} + \varepsilon \mathbf{z}_k)$. Instead of jumping to the candidate directly, it is expressed in the shared update rule through the pseudo-gradient

$$\hat{\mathbf{g}}_{\text{RS}} = \begin{cases} \dfrac{\mathcal{L}_{k^*} - \mathcal{L}_0}{\varepsilon} \mathbf{z}_{k^*} & \text{if } \mathcal{L}_{k^*} < \mathcal{L}_0, \\ \mathbf{0} & \text{otherwise,} \end{cases} \tag{3}$$

which points along the improving direction (the difference is negative) and is zero when no candidate improves on the clean loss; Adam then takes a step of magnitude $\approx \varepsilon$ along $\mathbf{z}_{k^*}$. The acceptance rate (fraction of steps with a non-zero update) is logged. Adam's moment estimates are updated on rejected steps as well (with a zero gradient), so a rejected step still moves the parameters along the decaying momentum.

Node perturbation (NP). Classic node perturbation [Widrow and Lehr, 1990, Werfel et al., 2003, Fiete and Seung, 2006, Hiratani et al., 2022] perturbs the units rather than the weights. For layer l with input $\mathbf{h}_{l-1}$, pre-activation $\mathbf{a}_l = \mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l$ and post-activation $\mathbf{h}_l = f(\mathbf{a}_l)$, a noisy forward pass injects Gaussian noise into *every* layer and propagates it:

$$\tilde{\mathbf{a}}_l = \mathbf{W}_l \tilde{\mathbf{h}}_{l-1} + \mathbf{b}_l + \sigma \boldsymbol{\xi}_l, \qquad \boldsymbol{\xi}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \tilde{\mathbf{h}}_l = f(\tilde{\mathbf{a}}_l), \tag{4}$$

with $\sigma = \varepsilon$. Writing $\delta\mathcal{L}_n = \tilde{\ell}_n - \ell_n$ for the change of the per-example loss, the estimator for the weights and biases of layer l is

$$\hat{\mathbf{g}}^{\text{NP}}_{\mathbf{W}_l} = \frac{1}{N} \sum_{n=1}^{N} \frac{\delta\mathcal{L}_n}{\sigma} \boldsymbol{\xi}_{l,n} \mathbf{h}_{l-1,n}^{\top}, \qquad \hat{\mathbf{g}}^{\text{NP}}_{\mathbf{b}_l} = \frac{1}{N} \sum_{n=1}^{N} \frac{\delta\mathcal{L}_n}{\sigma} \boldsymbol{\xi}_{l,n}, \tag{5}$$

i.e. the outer product of the noise-weighted loss change with the *clean* layer input (for convolutions, the matching correlation over unfolded patches; for embeddings, the one-hot input). With one clean pass and 15 noisy passes the estimator is averaged over 15 independent noise draws. Note that $\delta\mathcal{L}_n/\sigma \cdot \boldsymbol{\xi}_l$ equals $\delta\mathcal{L}_n\, \varepsilon_l/\sigma^2$ with $\varepsilon_l = \sigma\boldsymbol{\xi}_l$, the form used by Dalm et al. [2023]; the implementation uses exactly Eq. (5) with the batch average over the per-example loss changes, so that $\hat{\mathbf{g}}$ estimates the gradient of the batch-mean loss on the same scale as backpropagation. For a layer applied several times per forward pass (the LSTM gate layer, once per time step) the same $\delta\mathcal{L}_n$ weights every call.

Activity-based node perturbation (ANP). Because the noise injected in early layers propagates, the actual perturbation of layer l in Eq. (4) is not $\sigma\boldsymbol{\xi}_l$ but the measured activity difference $\delta\mathbf{a}_l = \tilde{\mathbf{a}}_l - \mathbf{a}_l$. Activity-based NP [Dalm et al., 2023] replaces the injected noise by this measured difference and normalises by the norm of the total activity perturbation across all layers:

$$\hat{\mathbf{g}}^{\text{ANP}}_{\mathbf{W}_l} = \frac{1}{N} \sum_{n=1}^{N} c\, \frac{\delta\mathcal{L}_n}{\|\delta\mathbf{a}_n\|^2}\, \delta\mathbf{a}_{l,n} \mathbf{h}_{l-1,n}^{\top}, \qquad \|\delta\mathbf{a}_n\|^2 = \sum_{l'=1}^{L} \|\delta\mathbf{a}_{l',n}\|^2, \tag{6}$$

where c is a normalisation constant ($c = \sum_l N_l$, the total number of perturbed units, in Dalm et al. [2023]), with the bias analogue obtained by dropping $\mathbf{h}_{l-1,n}^{\top}$. For a single perturbed unit the estimator

reduces to Eq. (5); in general it corrects for the propagated perturbation that classic NP ignores. Following the authors' reference implementation, $c = \sum_l N_l$ counts every perturbed pre-activation unit of the network (all layers, all time steps or positions; the input is not counted) and the norm is taken per example over the same set of units.

Iterative node perturbation (INP). Iterative NP [Dalm et al., 2023] removes the propagation problem differently: each noisy pass perturbs *one* layer only, so that the loss change $\delta\mathcal{L}_n^{(l)}$ is attributable to that layer and the estimator is a directional derivative along the injected noise $\mathbf{v}_{l,n} = \sigma\boldsymbol{\xi}_{l,n}$,

$$\hat{\mathbf{g}}_{\mathbf{W}_l}^{\text{INP}} = \frac{1}{N}\sum_{n=1}^{N} N_l \,\frac{\delta\mathcal{L}_n^{(l)}}{\|\mathbf{v}_{l,n}\|^2}\,\mathbf{v}_{l,n}\,\mathbf{h}_{l-1,n}^{\top}, \tag{7}$$

with N_l the number of perturbed units of layer l (Eq. 3 of Dalm et al. [2023]), at the price of one pass per layer: with L perturbable layers, L passes are needed for one estimate of the full gradient. The 15 noisy passes of the budget give $\lfloor 15/L \rfloor$ draws per layer, i.e. 5 for the MLP and the LSTM ($L = 3$), 3 for the ConvNet ($L = 5$) and exactly 1 for the Transformer ($L = 15$: two embeddings, twelve projection layers of the two blocks and the readout), so every architecture uses 16 passes (Table 1).

Decorrelated ANP (DANP). Decorrelated ANP [Dalm et al., 2023] is ANP in which the input of every layer is first decorrelated by a square matrix $\mathbf{R}_l$ that is *not* a parameter of the model: $\hat{\mathbf{h}}_{l-1} = \mathbf{R}_l\mathbf{h}_{l-1}$, $\mathbf{a}_l = \mathbf{W}_l\hat{\mathbf{h}}_{l-1} + \mathbf{b}_l$, with $\mathbf{R}_l$ initialised to the identity and trained by the local rule of Ahmad et al. [2023],

$$\mathbf{R}_l \leftarrow \mathbf{R}_l - \eta_R\left(\hat{\mathbf{C}}_l - \operatorname{diag}\hat{\mathbf{C}}_l\right)\mathbf{R}_l, \qquad \hat{\mathbf{C}}_l = \frac{1}{M}\sum_{m=1}^{M}\hat{\mathbf{h}}_{l-1,m}\hat{\mathbf{h}}_{l-1,m}^{\top}, \tag{8}$$

where $\hat{\mathbf{C}}_l$ is the batch covariance of the decorrelated inputs recorded on the clean pass (M rows: examples, or examples $\times$ positions for sequence and convolutional layers, whose decorrelated vector is the unfolded $C_{\text{in}}k^2$ patch). The ANP estimator (6) is then applied with $\hat{\mathbf{h}}_{l-1}$ in place of $\mathbf{h}_{l-1}$. Embedding layers, whose inputs are one-hot, are not decorrelated. The decorrelation rate is fixed at $\eta_R = 10^{-3}$ (the value of Dalm et al. [2023], Appendix D, and the default of their code) and is not tied to ε; $\mathbf{R}_l$ is updated once per step, after the Adam step, from the inputs recorded on that step's clean pass. The $\mathbf{R}_l$ add no capacity (they compose linearly with $\mathbf{W}_l$) and are not counted in the parameter budget.

Forward-forward (FF). The forward-forward algorithm [Hinton, 2022] trains each layer with a local objective and never propagates an error signal between layers. For a layer with activity $\mathbf{h}_l$ the goodness is $G = \operatorname{mean}_j h_{l,j}^2$, and the local loss on a positive/negative pair is

$$\ell_{\text{FF}} = \log\!\left(1 + e^{\,\theta - G_{\text{pos}}}\right) + \log\!\left(1 + e^{\,G_{\text{neg}} - \theta}\right), \qquad \theta = 2, \tag{9}$$

whose gradient with respect to the layer's own parameters is written into the gradient slot (local backward within the layer only). Activities are ℓ_2-normalised before being passed to the next layer so that only their direction, not their goodness, is forwarded. For classification, positive data are inputs with the correct label overlaid on the first 10 pixels and negative data carry a wrong label; a softmax readout, trained with the same optimizer on the normalised top-layer activity of a pass with a neutral (uniform) label, provides the test loss and accuracy. For character-level language modelling, positive sequences are the real text, negative sequences have 30 % of their characters replaced at random, and the readout predicts the next character at every position from the normalised top-layer activity. The positive, negative and neutral-label inputs are processed together as one batch of $3N$ examples, so a step costs one positive and one negative pass over the blocks; the readout is trained on detached features and therefore never shapes the hidden layers. A single Adam instance covers all parameters: every parameter receives exactly one local gradient per step. Because the LSTM output $\sigma(o)\tanh(c)$ lies in $(-1, 1)$, its goodness cannot exceed 1 and the threshold $\theta = 2$ would saturate every unit for positive and negative data alike; for that block we use $\theta = 0.25$, a quarter of the attainable range. We report both the readout accuracy and Hinton's label-accumulation accuracy (the label whose overlay yields the highest total goodness).

Table 1: How each method spends its forward-pass budget per step. L is the number of perturbable layers (3 for the MLP and the LSTM, 5 for the ConvNet, 15 for the Transformer).

Method	Clean passes	Probe passes	Backward	Estimates per step
BP	1	0	1 full	exact gradient
SPSA	0	16	–	8 central differences, averaged
RS	1	15	–	best of 15 candidates
NP	1	15 (all layers)	–	15 noise draws, averaged
ANP	1	15 (all layers)	–	15 noise draws, averaged
INP	1	15 (one layer each)	–	$15/L$ draws per layer
DANP	1	15 (all layers)	–	15 noise draws, averaged; local $\mathbf{R}_l$ update
FF	2 (pos. + neg.)	0	per layer, local	local goodness gradients

Table 2: Tasks, data splits (train/validation/test) and architectures. All hidden nonlinearities are ReLU; convolutions are 3×3 with 2×2 max-pooling. The LSTM [Hochreiter and Schmidhuber, 1997] has one layer of 80 units; the Transformer [Vaswani et al., 2017] uses two pre-LN blocks, model width 48, 4 heads, feed-forward width 64, learned positional embeddings and context length $T = 64$. V is the character vocabulary size. *Units* is the number of pre-activation units one node-perturbation probe perturbs for a single example, summed over layers, time steps and spatial positions; it is the effective search dimension of the NP family, as P is for SPSA and RS.

Key	Data	Split	Model	Params	Units
`mlp_mnist`	MNIST	55k / 5k / 10k	MLP 784–48–32–10	39,578	90
`conv_cifar10`	CIFAR-10	45k / 5k / 10k	conv16–conv32–conv32–fc48–10	39,450	26,682
`lstm_ptb`	PTB (char)	standard	emb32–LSTM80–V	41,810	25,728
`transformer_ptb`	PTB (char)	standard	emb48 + pos64–2 blocks–V	39,250	45,232

Backpropagation (reference). One forward and one backward pass produce the exact gradient $\hat{\mathbf{g}}_{\text{BP}} = \nabla_{\boldsymbol{\theta}}\mathcal{L}$, which is fed to the same Adam update with the same ε. It is the upper reference, not a zero-order method.

3.3 Forward-pass budget

Every probe-based method is given exactly 16 forward passes per step. The passes are spent differently (Table 1): SPSA uses all 16 for 8 antithetic pairs and has no clean pass; RS and the node-perturbation family use one clean pass to record the clean activities (and, for RS, the clean loss) plus 15 probe passes. Forward-forward uses two passes (positive and negative) and backpropagation one forward plus one backward pass; they are not probe methods and are included as references for the two ends of the spectrum. Because the number of passes per step is fixed, the number of steps per hour differs between methods (Section 5.4).

4 Experimental Setup

Tasks and architectures. Table 2 lists the four tasks. All models have ≈40k parameters so that a run fits comfortably within the budget and the methods are compared at the same scale. Weights are Xavier-uniform initialised [Glorot and Bengio, 2010] with zero biases (the LSTM forget-gate bias is set to 1); layer normalisation [Ba et al., 2016] in the Transformer carries no affine parameters, so that every parameter of every model lives in a perturbable layer. MNIST [LeCun et al., 1998] and CIFAR-10 [Krizhevsky, 2009] use the last 5k images of the official training sets as validation; inputs are standardised (per channel for CIFAR-10). Penn Treebank [Marcus et al., 1993] is used at the character level [Mikolov et al., 2012] with the standard train/valid/test files and a vocabulary built from the training text; a word-level embedding would by itself exceed the 40k-parameter budget. Classification uses batches of 128 images; language modelling uses batches of 64 sequences of 64 characters. There is no augmentation, dropout or weight decay.

The character-level Penn Treebank corpus is built from the standard Mikolov word-level files by joining lines with a newline and keeping spaces; its vocabulary has $V = 50$ symbols (5.10M / 0.40M / 0.45M characters in train / valid / test).

Optimization and sweep. Every method uses Adam with $(\beta_1, \beta_2) = (0.9, 0.99)$ and learning rate ε, with ε also the probe scale (Section 3.1). We sweep $\varepsilon \in \{0.1,\ 0.01,\ 0.001\}$, held fixed for the whole run. The loss is cross-entropy: per example for classification, per sequence (mean over the 64 next-character predictions) for language modelling. Zero-order estimates use per-example loss differences where the method allows it (the node-perturbation family) and batch-mean losses where it does not (SPSA, RS).

Budget and evaluation. Each run trains for **one hour of wall-clock** on a single NVIDIA RTX 5090; evaluation time is excluded from the clock. Validation and test loss (and accuracy or bits-per-character) are evaluated at initialisation, every 5 minutes of training time and at the end. A run is stopped and marked as diverged when a loss becomes NaN, or when at two consecutive evaluations its training loss exceeds $2\times$ its initial value or its validation loss exceeds $2\times$ its value at initialisation.

Model selection and reporting. For each (task, optimizer) pair we select the ε with the lowest final *validation* loss and report that run's final *test* loss, together with test accuracy for classification and bits-per-character ($\text{loss}/\ln 2$) for language modelling. The grid is 4 tasks $\times$ 8 optimizers $\times$ 3 values of ε = **96 runs**, executed 8 at a time on 8 GPUs (one run per GPU). Every run is logged to Weights & Biases (project `zoo-survey`, run name `{task}-{opt}-lr{eps}`) and writes a JSON file with its configuration, history and final metrics.

Hardware and software. All runs use one RTX 5090 (32 GB) per run on a single 8-GPU node (Intel Xeon Platinum 8490H, NVIDIA driver 580.173, CUDA 12.8, PyTorch 2.9.1, Python 3.11) rented on vast.ai, and the code at `https://github.com/Fchaubard/zero-order-ablations` (commit `db100bc` for the first ten runs and `f5424e9` for the other 86 and the supplementary runs; the latter adds the validation-loss clause of the divergence rule, Section 6).

5 Results

5.1 Main comparison

Table 3 reports, for every task and optimizer, the final test loss of the run selected by validation loss, with test accuracy (classification) or bits-per-character (language modelling) as the second number. The best zero-order method in each row is set in bold; backpropagation and forward-forward are references. Among the probe methods, ANP has the lowest final test loss on both image tasks (0.381 on MNIST, 96.4 % accuracy; 1.177 on CIFAR-10, 59.8 %), followed by DANP (0.751 / 94.2 % and 1.285 / 60.4 %) and INP (0.916 / 97.0 % and 1.339 / 52.7 %), while INP is the best probe method on both sequence tasks (1.955, i.e. 2.82 bits/char, on the LSTM and 2.151, 3.10 bits/char, on the Transformer), ahead of NP and SPSA on the LSTM and of SPSA and RS on the Transformer. Against backpropagation the picture splits by task family. On the sequence tasks BP reaches 1.78 and 1.82 bits/char, so the best zero-order rule trails it by 1.0 and 1.3 bits/char, a factor 1.6–1.7 in test loss. On the image tasks the *final* BP numbers (1.857 / 20.9 % on MNIST, 2.402 / 50.2 % on CIFAR-10) are worse than those of the node-perturbation methods, because backpropagation completes 3–5 million steps in the hour, overfits the 40k-parameter models within minutes and then drifts: its $\varepsilon = 0.001$ MNIST run ends at 97.4 % accuracy but a test loss of 2.355, so validation loss selects the $\varepsilon = 0.1$ run that never left 20.9 % (Appendix A), and its $\varepsilon = 0.001$ CIFAR-10 run was stopped by the divergence rule at 67.4 % accuracy with a validation loss ten times its initial value (Section 6).

Table 4 therefore complements the protocol's final numbers with an early-stopping oracle: for every (task, optimizer) pair it reports the evaluation, over all three ε and all 5-minute evaluations, with the lowest validation loss. Under this view BP recovers its expected standing on the image tasks (97.4 % on MNIST at minute 5; with 30-second evaluations it reaches 97.5 % with test loss 0.26 after 30 seconds, and 69.7 % on CIFAR-10 after 30 seconds, Section 6), ANP wins MNIST (0.137, 96.7 %), DANP wins CIFAR-10 (1.085, 62.9 %, reached after five minutes), and the sequence-task entries barely change because every probe method there is still improving at minute 60. The ranking of the probe methods is stable across the two views: INP has the best mean rank (2.0 under the final-loss protocol, 1.75 under early stopping), NP, DANP, SPSA and ANP share the middle (3.25–4.25), and RS and FF are last.

Table 3: Main result: final test loss after one hour of training, with test accuracy (%) for MNIST / CIFAR-10 or bits-per-character for PTB in the second row of each block. Each entry is the run with the lowest final validation loss over $\varepsilon \in \{0.1, 0.01, 0.001\}$. The best zero-order probe method per row is in bold; BP is the reference and FF is a local (non-probe) rule. A dagger marks a diverged run. For FF the accuracy of label-accumulation inference is given in parentheses. On the image tasks the final BP entries are dominated by overfitting (Section 5.1); Table 4 gives the early-stopped values.

		BP	Zero-order probe methods						FF
Task	Metric		SPSA	RS	NP	ANP	INP	DANP	
MNIST / MLP	test loss	1.857	1.039	1.412	2.141	**0.381**	0.916	0.751	2.127
	acc. (%)	20.9	72.2	62.0	96.8	**96.4**	97.0	94.2	19.1 (96.0)
CIFAR-10 / ConvNet	test loss	2.402	2.468	2.406	1.613	**1.177**	1.339	1.285	1.886
	acc. (%)	50.2	12.0	9.4	41.9	**59.8**	52.7	60.4	33.0 (32.7)
PTB / char-LSTM	test loss	1.231	2.048	2.124	2.015	3.232	**1.955**	2.608	2.688
	bits/char	1.78	2.96	3.06	2.91	4.66	**2.82**	3.76	3.88
PTB / char-Transformer	test loss	1.262	2.181	2.279	2.357	3.412	**2.151**	2.525	2.959
	bits/char	1.82	3.15	3.29	3.40	4.92	**3.10**	3.64	4.27

Table 4: Early-stopping oracle: test loss and test accuracy (%; in parentheses for FF: label-accumulation inference) or bits per character at the evaluation with the lowest *validation* loss over all ε and all evaluation times; the third row of each block gives that ε and the minute of training at which it occurred (ε/min; minute 0 = initialisation, i.e. no improvement). Best probe method per task in bold.

		BP	Zero-order probe methods						FF
Task	Metric		SPSA	RS	NP	ANP	INP	DANP	
MNIST / MLP	test loss	1.260	0.391	0.487	0.167	**0.137**	0.162	0.321	1.051
	acc. (%)	97.4	88.8	86.3	96.6	**96.7**	96.3	93.2	61.9 (93.2)
	ε/min	0.001/5	0.001/5	0.001/5	0.001/5	0.001/5	0.001/5	0.01/5	0.001/5
CIFAR-10 / ConvNet	test loss	1.319	1.642	1.742	1.434	1.139	1.341	**1.085**	1.819
	acc. (%)	56.0	40.3	34.9	48.8	61.7	53.1	**62.9**	34.8 (24.4)
	ε/min	0.01/45	0.001/5	0.001/10	0.001/10	0.001/50	0.001/45	0.001/5	0.001/20
PTB / char-LSTM	test loss	1.231	2.048	2.124	2.015	2.969	**1.955**	2.608	2.658
	bits/char	1.78	2.96	3.06	2.91	4.28	**2.82**	3.76	3.84
	ε/min	0.001/60	0.001/60	0.001/60	0.001/60	0.001/25	0.001/60	0.001/60	0.01/20
PTB / char-Transformer	test loss	1.260	2.181	2.279	2.342	3.065	**2.151**	2.513	2.925
	bits/char	1.82	3.15	3.29	3.38	4.42	**3.10**	3.63	4.22
	ε/min	0.001/55	0.001/60	0.001/60	0.001/35	0.001/10	0.001/60	0.01/50	0.01/25

5.2 Training curves

Figure 1 shows test loss against training wall-clock for the selected ε of every method on every task. Plotting against time rather than steps is the fair comparison under our protocol: methods that spend 16 passes per step take proportionally fewer steps per hour, and sequence models pay for every pass with $T = 64$ sequential time steps. The curves have three shapes. On MNIST every probe method reaches its minimum at the first evaluation (minute 5: ANP 0.137, INP 0.162, NP 0.167) and the test loss then rises for the rest of the hour while the accuracy barely moves (NP: 96.6 % to 96.8 % as the loss climbs from 0.167 to 2.141), i.e. the networks become over-confident rather than wrong; the same happens to BP within its first minute (Section 6). On CIFAR-10 the methods separate: DANP drops to 1.09 within five minutes and then creeps up, ANP and INP improve slowly for the whole hour (ANP reaches its minimum at minute 50), NP bottoms out at minute 10 and drifts, and SPSA, RS and FF stay near the chance loss of 2.30. On both sequence tasks every probe method is still improving at minute 60, slowly, on a plateau roughly one bit per character above BP, which is itself still improving; the Transformer panel shows the spikes of ANP, whose updates are erratic on sequence models (Section 5.5). Appendix C shows the curves of all 96 runs: at $\varepsilon = 0.1$ most probe methods diverge within the first two evaluations, and at $\varepsilon = 0.01$ several reach a good minimum early and then blow up (INP on MNIST, NP on CIFAR-10, ANP and DANP on the LSTM).

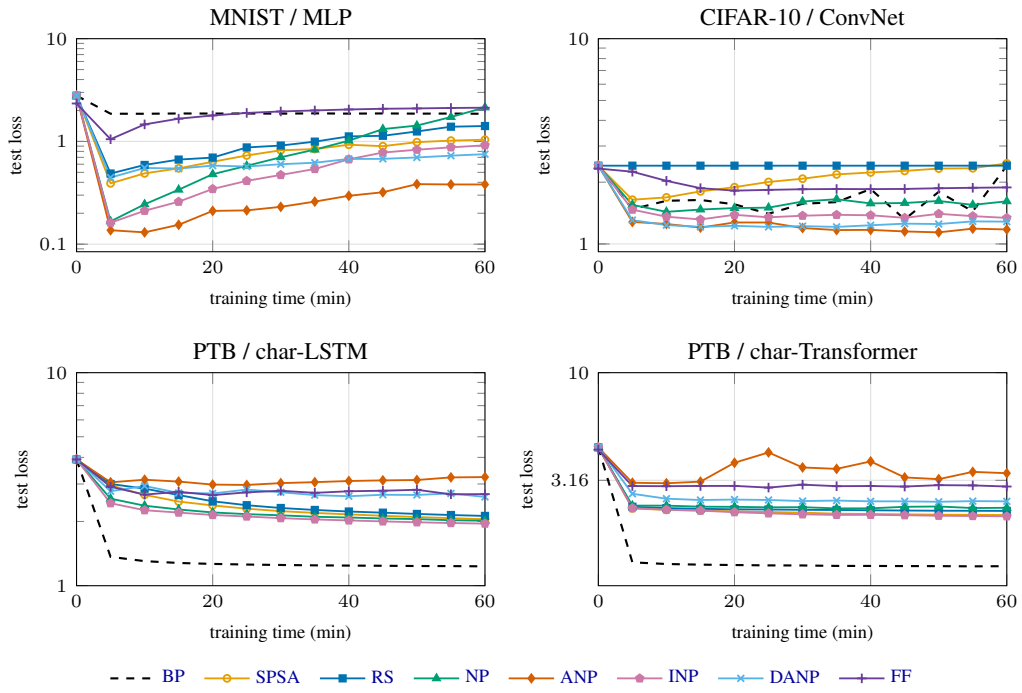


Figure 1: Test loss versus training time (minutes, evaluation time excluded) for the ε selected by validation loss, one line per optimizer. Log-scale y-axis, capped at 10. BP is the black dashed reference; on MNIST its selected run is $\varepsilon = 0.1$, which never leaves 20.9 % accuracy, because the $\varepsilon = 0.001$ run overfits to a higher final loss at 97.4 % accuracy (Appendix C).

5.3 Sensitivity to the probe scale

Because ε is both the learning rate and the probe scale, its sweep probes two things at once: how large a step each method can tolerate and how small a probe each method can resolve. Figure 2 plots the final test loss of every run against ε. The sweep is decisive. At $\varepsilon = 0.1$, 15 of the 24 probe-method runs diverge (all four DANP runs, three each of NP, ANP and RS, two of SPSA, none of INP) and the nine survivors sit at chance or above the uniform loss; BP still trains the language models at this step size (2.57 and 2.65 bits/char) but collapses on the image tasks, and FF never diverges but its readout is at chance. At $\varepsilon = 0.01$ seven probe runs diverge, five of them node-perturbation runs, and BP itself is stopped on MNIST. At $\varepsilon = 0.001$ no probe run diverges, 25 of the 32 (task, method) pairs select this value, and only SPSA and RS on CIFAR-10 (12.0 and 9.4 %) and the FF readout on MNIST fail to move. Two patterns stand out. First, the node-perturbation family tolerates less noise than the weight-space methods: it accounts for 10 of the 15 divergences at $\varepsilon = 0.1$ and 5 of 7 at $\varepsilon = 0.01$, consistent with the node noise entering every layer's pre-activations directly, and DANP is the most fragile of all (Section 6). Second, INP is the only probe method with no divergence at $\varepsilon = 0.1$; its single flag (MNIST, $\varepsilon = 0.01$) was raised at the final evaluation, after an hour of oscillation around 70 % accuracy.

5.4 Throughput

Table 5 lists the number of optimizer steps and forward passes completed per hour by every method on every task. Since the per-step budget is 16 passes for the probe methods, 2 for FF and 1 (plus a backward) for BP, the step rate is the quantity that differs across methods, while the pass rate shows how efficiently each implementation uses the GPU (batched probes, in-place perturbation, sequential LSTM time steps). Sixteen batched probe passes cost 4.3–4.7 backpropagation steps on every task (SPSA completes 1.09M, 0.62M, 34k and 219k steps per hour on the four tasks against 4.94M, 2.93M, 148k and 1.02M for BP), so one probe pass costs about 0.28 of a forward-plus-backward and the probe methods see 3.4–3.7 times more forward passes per hour than BP: the much lower step count is the price of the estimator, not of the implementation. Within the probe family, NP is 1.5–1.7 times

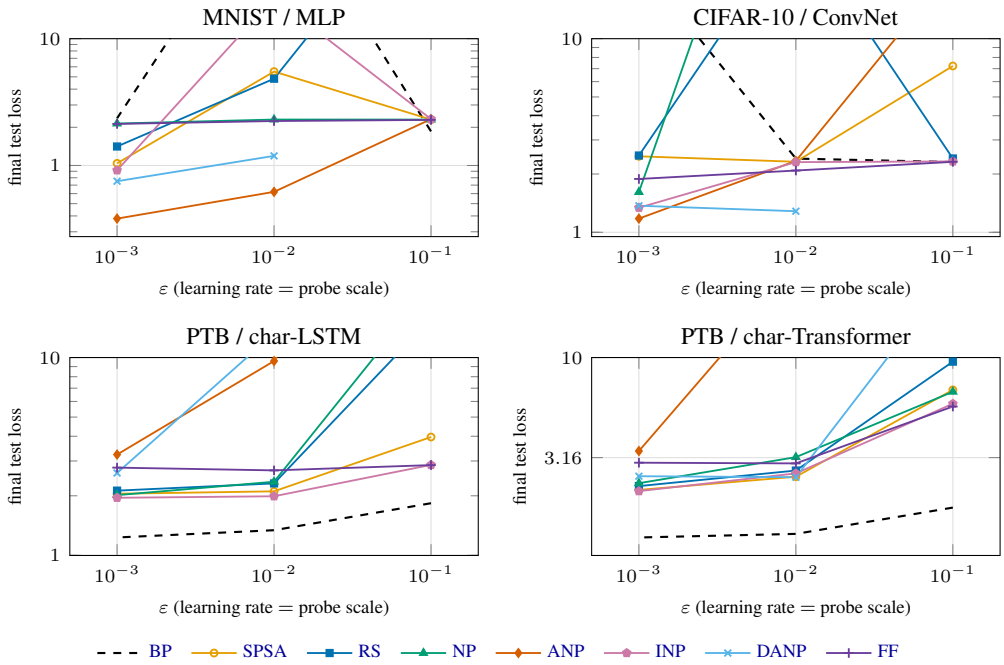


Figure 2: Final test loss versus ε (learning rate = probe scale) for every optimizer and task; all 96 runs. Diverged runs are plotted at their last finite evaluation.

Table 5: Throughput: optimizer steps per hour and forward passes per hour (in thousands) for every method and task, measured on one RTX 5090 at the selected ε.

Task	Metric	BP	SPSA	RS	NP	ANP	INP	DANP	FF
MNIST / MLP	steps/h (k)	4944.5	1094.9	1142.2	645.0	475.9	633.7	383.5	1840.9
	fwd passes/h (k)	4944.5	17518.5	18274.7	10320.1	7614.3	10139.4	6136.4	3681.8
CIFAR-10 / ConvNet	steps/h (k)	2933.8	623.6	647.1	373.8	274.3	415.3	218.0	1106.0
	fwd passes/h (k)	2933.8	9977.5	10353.9	5980.5	4388.8	6644.8	3487.5	2211.9
PTB / char-LSTM	steps/h (k)	147.5	34.3	34.5	23.6	19.8	24.9	17.0	139.7
	fwd passes/h (k)	147.5	548.1	552.7	377.4	317.2	398.9	272.5	279.4
PTB / char-Transformer	steps/h (k)	1021.0	219.1	229.1	125.7	101.0	168.0	86.8	777.6
	fwd passes/h (k)	1021.0	3506.3	3665.6	2010.5	1615.2	2688.0	1388.1	1555.2

slower than SPSA because its clean pass builds a local graph per layer and runs one short backward per layer; ANP adds 20–36 % for the per-example activity norms; DANP adds another 16–26 % for the decorrelation update and is 1.4–1.7 times slower than NP (2.9 times slower than SPSA on the MLP); INP is as fast as NP on the MLP and up to 34 % faster on the Transformer because each noisy pass perturbs one layer only. FF takes 2.7 times fewer steps than BP on the image models despite its two-pass budget, because it forwards a triple batch (positive, negative and neutral) and runs one backward per block, but matches BP on the LSTM, whose cost is dominated by the sequential time steps. The LSTM is the slowest model for every method: at $T = 64$ sequential steps per pass, SPSA completes 32 times fewer steps per hour than on the MLP, and the probe methods manage only 17k–35k steps in the hour.

5.5 Analysis

Which zero-order method wins? INP is the answer if a single rule must be chosen: it is the best probe method on both sequence tasks, second or third in test loss on both image tasks with the highest final MNIST accuracy of any probe method (97.0 %), has the best mean rank under both views of the results (2.0 and 1.75 of 7) and is the only probe method that survives $\varepsilon = 0.1$ on every task. ANP is the best rule on the image tasks under the protocol (0.381 and 1.177, against INP's 0.916 and 1.339)

and DANP is the fastest to a good CIFAR-10 model (62.9 % after five minutes), but both fail on the sequence models, where ANP is the worst probe method (4.66 and 4.92 bits/char) and DANP fifth. The gap to backpropagation depends on the task family. On the image tasks it is small at the best evaluation (96.7 versus 97.5 % on MNIST, 62.9 versus 69.7 % on CIFAR-10) and the protocol's final numbers even favour the probe methods, which take 7–13 times fewer steps than BP and therefore do not overfit within the hour. On the sequence tasks the best probe method ends 1.0 (LSTM) and 1.3 (Transformer) bits per character behind BP, a factor 1.6–1.7 in test loss, and is still on a slowly descending plateau at minute 60.

Weight probes versus node probes. SPSA and RS perturb all $P \approx 40$k weights at once and obtain one scalar loss difference per probe, so the variance of their estimate grows with P and 16 passes yield a direction whose cosine with the gradient is of order $\sqrt{16/P} \approx 0.02$. The node-perturbation family obtains a loss difference *per example* (128 or 64 per pass) and assigns credit per unit, so its effective dimension is the number of perturbed units (Table 2): 90 on the MLP against 39,578 weights. The results track this ratio. On MNIST the node methods reach 96–97 % while SPSA and RS reach 72 and 62 % (89 and 86 % at their best evaluation). On the ConvNet the 26,682 perturbed unit-positions are of the same order as the 39,450 weights and plain NP drops to 41.9 %; we measured the cosine between the NP estimate and the true gradient on this network and found it at 0.0 for the first two convolutional layers with 480 probes (0.55 for the third, 0.94 and 0.99 for the dense layers), rising to 0.59 only with 7,680 probes, so with 15 probes per step the early layers receive essentially no signal and Adam, which normalises every coordinate, turns the noise into a random walk of fixed size ε. On the sequence models the number of unit-positions (25,728 and 45,232) exceeds P, and the two families end within 0.1–0.3 bits/char of each other (SPSA 2.96 and 3.15, INP 2.82 and 3.10), with RS close behind. The weight-space methods are, however, markedly more tolerant of large ε and 1.5–2.9 times faster per step.

The effect of activity-based, iterative and decorrelated variants. ANP's activity-based attribution clearly helps on the feed-forward image models: it beats NP by a wide margin in final loss (0.381 versus 2.141 on MNIST, 1.177 versus 1.613 on CIFAR-10) and in accuracy on CIFAR-10 (59.8 versus 41.9 %), in line with Dalm et al. [2023]. On the recurrent and attention models it is a liability: ANP diverges at $\varepsilon \geq 0.01$ on both and barely leaves the unigram level at $\varepsilon = 0.001$ (4.66 and 4.92 bits/char against 2.91 and 3.40 for NP). The estimator itself is sound there—on the LSTM its cosine with the true gradient is 0.45–0.82 with 480 probes, close to NP's 0.55–0.95—but its normalisation by the squared activity difference, summed over 25k–45k unit-positions whose perturbations propagate through shared weights, produces updates of erratic magnitude (the spikes in Figure 1). INP's one-layer-at-a-time attribution has only $15/L$ draws per layer (5, 3, 5 and 1 on the four models), yet removes all cross-layer noise from each estimate, and this trade is worth it on every task: INP is never worse than third and is the best rule on both sequence models, including the Transformer with a single draw per layer. Decorrelation helps where Dalm et al. [2023] reported it and not elsewhere: DANP is the fastest method on CIFAR-10 (1.085 at minute 5, a level ANP needs 50 minutes to reach) and improves on ANP on both sequence models (3.76 and 3.64 versus 4.66 and 4.92 bits/char), but ends behind ANP on CIFAR-10 after the full hour (1.285 versus 1.177) and clearly behind on MNIST (0.751 versus 0.381, 94.2 versus 96.4 %); it is also the most fragile rule (five of its twelve runs diverged, including NaN within 21 steps at $\varepsilon = 0.1$ on both image models, Section 6) and the slowest (Table 5). Decorrelation is not applied to the embedding layers, so on the sequence models it acts on the recurrent / attention and output layers only.

Sequence models. The sequence models are the hardest setting for every zero-order rule and the most expensive: a pass through the LSTM is 64 sequential time steps, so the probe methods complete only 17k–35k steps in the hour (Table 5). On the test split, a uniform model scores 5.64 bits/char, a unigram 4.32, a bigram 3.31 and an add-0.1 trigram 2.61. Every probe method beats the unigram on both models; on the LSTM, INP, NP, SPSA and RS (2.82–3.06) beat the bigram and sit between bigram and trigram, DANP (3.76) and FF (3.88) do not, and ANP (4.66) is near the unigram; on the Transformer only INP, SPSA and RS (3.10, 3.15 and 3.29) edge past the bigram. BP reaches 1.78 and 1.82 bits/char, well beyond the trigram, so after one hour the zero-order rules have learned roughly bigram statistics while backpropagation has learned a real character model. All probe curves are still descending at minute 60 (Figure 1), so part of this gap is budget rather than estimator, but at 17k–35k steps per hour closing it would take days. FF with corrupted-sequence negatives trains fast (140k and 778k steps per hour) and its goodness does separate real from corrupted text, but the normalised

features support only a weak next-character readout (3.88 and 4.27 bits/char, between unigram and bigram).

Forward-forward. FF is not a probe method: it takes 1.8M steps per hour on the MLP (2.9 times NP's count, 37 % of BP's), uses exact local gradients within each block and never diverged at any ε. Its test loss, however, is set by the linear readout on the neutral-label features of the last block, and that readout is poor on the image tasks: 19.1 % on MNIST and 33.0 % on CIFAR-10, while Hinton's label-accumulation inference on the same trained blocks gives 96.0 % and 32.7 %. On MNIST the readout even degrades during training (61.9 % at minute 5, 19.1 % at minute 60) and its own training cross-entropy rises towards $\ln 10$, i.e. the neutral-label activity of a 32-unit block loses class information as the goodness objective is optimised; the readout follows Hinton's recipe, but that recipe assumed layers of 2,000 units. Label-accumulation accuracy is also the most ε-robust number in the study (92.2, 95.0 and 96.0 % on MNIST at $\varepsilon = 0.1$, 0.01 and 0.001). On the sequence models FF is the second worst rule (3.88 and 4.27 bits/char), ahead only of ANP.

6 Discussion and Limitations

Final loss, overfitting and the divergence rule. The protocol reports the loss after a fixed hour and selects ε by final validation loss. For backpropagation, which completes 3–5 million steps on the image models, this measures overfitting rather than optimisation: with 30-second evaluations (four supplementary 10-minute runs, released with the logs) BP reaches 97.5 % on MNIST and 69.7 % on CIFAR-10 after 30 seconds, and its test loss then grows while the accuracy holds. The divergence rule, which stops a run whose validation loss exceeds twice its initial value at two consecutive evaluations, was added to catch probe runs whose training loss is masked by the probe noise, and it also stopped three overfitting runs (BP on CIFAR-10 at $\varepsilon = 0.001$ and on MNIST at $\varepsilon = 0.01$, INP on MNIST at $\varepsilon = 0.01$); the first ten runs of the sweep, all at $\varepsilon = 0.1$, started before this clause existed, and none of them would have been stopped earlier by it. We keep the protocol numbers in Table 3 because they are what the pre-registered procedure produces, and report the early-stopping oracle (Table 4) alongside; weight decay, early stopping or a smaller learning rate for BP would make the reference stronger on the image tasks without changing the ranking of the zero-order rules.

DANP's decorrelation rate. The decorrelation matrices are updated with a fixed rate $\eta_R = 10^{-3}$ that is not normalised by the input scale. At $\varepsilon = 0.1$ a single Adam step inflates the ReLU activities of the hidden layers by an order of magnitude, the update $\mathbf{R}_l \leftarrow \mathbf{R}_l - \eta_R(\mathbf{C}_l - \operatorname{diag} \mathbf{C}_l)\mathbf{R}_l$ then grows $\mathbf{R}_l$ tenfold per step, and the run is NaN within 21 steps on both image models (the input-layer $\mathbf{R}$, whose inputs are bounded pixels, stays stable). A rate tied to the activity scale would remove this failure mode but is not part of the published rule.

Single seed. Each of the 96 configurations is run once. The reported differences are therefore single draws of a random process (initialisation, data order, probes); we do not report error bars, and small differences in Table 3 should not be read as rankings. The one-hour-per-run protocol was chosen so that the full grid runs in about 11 hours of wall-clock on 8 GPUs (100 GPU-hours); repeating it with several seeds is the most important follow-up.

One-hour budget and 40k parameters. The budget is deliberately small and identical for all methods, which favours methods that make progress quickly over methods that converge slowly but further. The ≈40k-parameter models are small by modern standards; the variance of weight-space estimators grows with the number of parameters, so the relative standing of SPSA and RS against the node-perturbation family may change at larger scale, and none of our conclusions should be extrapolated to models with orders of magnitude more parameters without new experiments.

ε tied to the learning rate. Using one scalar for the probe scale and the Adam learning rate removes a dimension from the sweep but also removes a degree of freedom: a method that would benefit from a small probe and a large step (or vice versa) is handicapped. The three-point sweep is coarse, and the best ε of a method may lie between grid points.

Wall-clock depends on implementation efficiency. Comparing against training time rewards implementations that batch probes well and penalises methods whose natural implementation is

sequential (iterative NP, the decorrelation update, LSTM time steps). We report forward passes per hour (Table 5) so that readers can re-weight the comparison by passes instead of time, but the ranking under a pass budget could differ.

FF and BP are not probe methods. Forward-forward uses local gradients within each layer and backpropagation uses the exact gradient; neither obeys the 16-pass budget, and both are included only as reference points for a local rule and for the ceiling.

Character-level PTB. Language modelling is done at the character level because a word-level embedding of the 10k-word PTB vocabulary would by itself exceed the parameter budget. Character-level results are not comparable to word-level perplexities in the literature.

7 Conclusion

We compared eight training rules—backpropagation, SPSA, random search, four node-perturbation variants and forward-forward—on four ≈40k-parameter architectures under one fully specified protocol: the same 16 forward passes per step, the same hour of training, the same Adam update and a single swept scalar that is both learning rate and probe scale. If one zero-order rule must be chosen, it is iterative node perturbation: it is the best probe method on both sequence models, within a point of the best on both image models, the most robust to the probe scale and the only variant with no divergence at $\varepsilon = 0.1$; activity-based and decorrelated node perturbation win the image tasks but fail on sequences, and weight-space perturbation is competitive only where the number of perturbed units approaches the number of weights. At their best evaluation the node-perturbation rules are within a point of backpropagation on MNIST and seven points on CIFAR-10, but on the character-level LSTM and Transformer the gap is still about one bit per character after an hour, with every zero-order curve on a slowly descending plateau. The protocol, code and all 96 run logs are public so that new zero-order rules can be added to the grid at the cost of 12 GPU-hours.

References

Nasir Ahmad. Correlations are ruining your gradient descent. *arXiv preprint arXiv:2407.10780*, 2024.

Nasir Ahmad, Ellen Schrader, and Marcel van Gerven. Constrained parameter inference as a principle for learning. *Transactions on Machine Learning Research*, 2023. arXiv:2203.13203.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Atılım Güneş Baydin, Barak A. Pearlmutter, Don Syme, Frank Wood, and Philip Torr. Gradients without backpropagation. *arXiv preprint arXiv:2202.08587*, 2022.

Aochuan Chen, Yimeng Zhang, Jinghan Jia, James Diffenderfer, Jiancheng Liu, Konstantinos Parasyris, Yihua Zhang, Zheng Zhang, Bhavya Kailkhura, and Sijia Liu. DeepZero: Scaling up zeroth-order optimization for deep model training. In *International Conference on Learning Representations (ICLR)*, 2024.

Xiangyi Chen, Sijia Liu, Kaidi Xu, Xingguo Li, Xue Lin, Mingyi Hong, and David Cox. ZO-AdaMM: Zeroth-order adaptive momentum method for black-box optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Sander Dalm, Marcel van Gerven, and Nasir Ahmad. Node perturbation can effectively train multi-layer neural networks. *arXiv preprint arXiv:2310.00965*, 2023.

Sander Dalm, Joshua Offergeld, Nasir Ahmad, and Marcel van Gerven. Efficient deep learning with decorrelated backpropagation. *arXiv preprint arXiv:2405.02385*, 2024.

John C. Duchi, Michael I. Jordan, Martin J. Wainwright, and Andre Wibisono. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015. doi: 10.1109/TIT.2015.2409256.

Ila R. Fiete and H. Sebastian Seung. Gradient learning in spiking neural networks by dynamic perturbation of conductances. *Physical Review Letters*, 97(4):048104, 2006. doi: 10.1103/PhysRevLett.97.048104.

Abraham D. Flaxman, Adam Tauman Kalai, and H. Brendan McMahan. Online convex optimization in the bandit setting: Gradient descent without a gradient. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 385–394, 2005.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, 2010.

Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

Nikolaus Hansen, Sibylle D. Müller, and Petros Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003. doi: 10.1162/106365603321828970.

Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

Naoki Hiratani, Yash Mehta, Timothy Lillicrap, and Peter E. Latham. On the stability and scalability of node perturbation learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1627–1635, 2017.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Timothy P. Lillicrap, Daniel Cownden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7: 13276, 2016. doi: 10.1038/ncomms13276.

Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero, and Pramod K. Varshney. A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications. *IEEE Signal Processing Magazine*, 37(5):43–54, 2020. doi: 10.1109/MSP.2020.3003837.

Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D. Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.

J. Matyas. Random optimization. *Automation and Remote Control*, 26(2):246–253, 1965.

Tomáš Mikolov, Ilya Sutskever, Anoop Deoras, Hai-Son Le, Stefan Kombrink, and Jan Černocký. Subword language modeling with neural networks. Preprint, http://www.fit.vutbr.cz/~imikolov/rnnlm/char.pdf, 2012.

Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017. doi: 10.1007/s10208-015-9296-2.

Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Leonard A. Rastrigin. The convergence of the random search method in the extremal control of a many parameter system. *Automation and Remote Control*, 24(10):1337–1342, 1963.

Mengye Ren, Simon Kornblith, Renjie Liao, and Geoffrey Hinton. Scaling forward gradient with local losses. In *International Conference on Learning Representations (ICLR)*, 2023.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

M. A. Schumer and Kenneth Steiglitz. Adaptive step size random search. *IEEE Transactions on Automatic Control*, 13(3):270–276, 1968. doi: 10.1109/TAC.1968.1098903.

James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi: 10.1109/9.119632.

James C. Spall. An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins APL Technical Digest*, 19(4):482–492, 1998.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Justin Werfel, Xiaohui Xie, and H. Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. In *Advances in Neural Information Processing Systems*, volume 16, pages 1197–1204, 2003.

Justin Werfel, Xiaohui Xie, and H. Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. *Neural Computation*, 17(12):2699–2718, 2005. doi: 10.1162/089976605774320539.

Bernard Widrow and Michael A. Lehr. 30 years of adaptive neural networks: Perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9):1415–1442, 1990. doi: 10.1109/5.58323.

Daan Wierstra, Tom Schaul, Tobias Glasmachers, Yi Sun, Jan Peters, and Jürgen Schmidhuber. Natural evolution strategies. *Journal of Machine Learning Research*, 15(27):949–980, 2014.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.

Yihua Zhang, Pingzhi Li, Junyuan Hong, Jiaxiang Li, Yimeng Zhang, Wenqing Zheng, Pin-Yu Chen, Jason D. Lee, Wotao Yin, Mingyi Hong, Zhangyang Wang, Sijia Liu, and Tianlong Chen. Revisiting zeroth-order optimization for memory-efficient LLM fine-tuning: A benchmark. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 59173–59190, 2024.

A Full 96-run grid

Table 6 lists the final validation and test loss of every run, for all three values of ε; the run selected for Table 3 in each row is the one with the lowest validation loss.

Table 6: Final validation / test loss of all 96 runs. For each (task, optimizer) row the ε with the lowest validation loss (in bold) is the run reported in Table 3. A dagger marks a diverged run (last finite evaluation shown).

		validation / test loss		
Task	Optimizer	$\varepsilon = 0.1$	$\varepsilon = 0.01$	$\varepsilon = 0.001$
MNIST / MLP	BP	**1.826 / 1.857**	51.107 / 177.402†	2.239 / 2.355
	SPSA	957.457 / 2.302†	2.193 / 5.502	**0.917 / 1.039**
	RS	52.924 / 170.573†	8.157 / 4.833	**1.238 / 1.412**
	NP	2.301 / 2.301†	2.303 / 2.302	**1.985 / 2.141**
	ANP	2.323 / 2.319	0.519 / 0.619	**0.363 / 0.381**
	INP	2.325 / 2.321	8.917 / 21.010†	**0.935 / 0.916**
	DANP	– / –†	1.033 / 1.190	**0.643 / 0.751**
	FF	2.288 / 2.288	2.243 / 2.241	**2.123 / 2.127**
CIFAR-10 / ConvNet	BP	2.305 / 2.305	**1.357 / 2.402**	23.735 / 24.475†
	SPSA	7.292 / 7.225†	22.978 / 2.313†	**2.300 / 2.468**
	RS	**2.395 / 2.406**	197.361 / 38.724†	2.401 / 2.492
	NP	1.9×10^6 / 1.3×10^6†	314.593 / 326.902†	**1.597 / 1.613**
	ANP	23.474 / 23.143†	2.332 / 2.336	**1.143 / 1.177**
	INP	2.327 / 2.323	2.303 / 2.303	**1.321 / 1.339**
	DANP	– / –†	**1.242 / 1.285**	1.289 / 1.371
	FF	2.311 / 2.311	2.081 / 2.084	**1.880 / 1.886**
PTB / char-LSTM	BP	1.858 / 1.835	1.371 / 1.340	**1.268 / 1.231**
	SPSA	3.991 / 3.965	2.125 / 2.107	**2.066 / 2.048**
	RS	15.802 / 15.689†	2.330 / 2.310	**2.141 / 2.124**
	NP	23.036 / 22.815†	2.380 / 2.356	**2.033 / 2.015**
	ANP	195.280 / 192.326†	9.592 / 9.603†	**3.242 / 3.232**
	INP	2.885 / 2.866	2.007 / 1.989	**1.971 / 1.955**
	DANP	271.941 / 271.567†	13.384 / 13.251†	**2.627 / 2.608**
	FF	2.862 / 2.859	**2.699 / 2.688**	2.789 / 2.776
PTB / char-Transformer	BP	1.802 / 1.780	1.351 / 1.316	**1.300 / 1.262**
	SPSA	6.865 / 6.893	2.557 / 2.539	**2.201 / 2.181**
	RS	9.453 / 9.538†	2.745 / 2.726	**2.297 / 2.279**
	NP	6.617 / 6.746	3.206 / 3.180	**2.374 / 2.357**
	ANP	305.304 / 306.216†	27.783 / 27.601†	**3.433 / 3.412**
	INP	5.921 / 5.880	2.651 / 2.638	**2.170 / 2.151**
	DANP	26.685 / 26.551†	**2.543 / 2.525**	2.564 / 2.554
	FF	5.707 / 5.694	**2.958 / 2.959**	2.976 / 2.983

B Implementation details and hyper-parameters

Perturbable layers. Every trainable layer of every model is a perturbable linear, convolutional or embedding layer. A global context selects what a layer does on its forward pass: (i) a plain differentiable forward (used by backpropagation, by the forward-forward local losses and by evaluation); (ii) a *clean* forward that detaches the layer input and records the input $\mathbf{h}_{l-1}$ and pre-activation $\mathbf{a}_l$, keeping a local autograd graph to the layer's own parameters only; or (iii) a graph-free *noisy* forward that adds $\sigma\boldsymbol{\xi}_l$ to the pre-activation of every layer (or of one selected layer for INP) and records $\boldsymbol{\xi}_l$ and $\tilde{\mathbf{a}}_l$. Given a per-unit credit tensor $\mathbf{G}_l$ (for NP, $\delta\mathcal{L}_n\boldsymbol{\xi}_{l,n}/\sigma$), the weight update $\mathbf{G}_l^\top\mathbf{h}_{l-1}$ and its convolutional and embedding analogues are obtained by a one-layer backward through the recorded local graph, so that no error is propagated through any nonlinearity. A layer that is called several times in one forward (the LSTM gate layer, once per time step) accumulates one record per call and the credit is summed over calls.

Decorrelation. For DANP, each linear and convolutional layer owns a square matrix $\mathbf{R}_l$ (initialised to the identity, stored as a buffer, not an optimizer parameter) that multiplies the layer input; for convolutions the decorrelated vector is the unfolded $C_{\text{in}}k^2$ patch. $\mathbf{R}_l$ is updated once per step with

Eq. (8) from the inputs recorded on the clean pass with $\eta_R = 10^{-3}$, from the very first step (no warm-up). Embedding layers are not decorrelated.

Language-model specifics. The LSTM is written with a single perturbable layer for the four gate pre-activations so that the gates are the perturbable nodes; the forget-gate bias is initialised to 1. The Transformer uses pre-LN blocks with parameter-free layer normalisation, a learned positional embedding over $T = 64$ positions, and a final parameter-free layer normalisation before the readout. The per-sequence loss is the mean next-character cross-entropy over the 64 positions.

Forward-forward specifics. One global Adam instance updates all parameters; since every block's goodness loss is computed on the detached, normalised output of the previous block, each parameter receives exactly one local gradient per step (the token and position embeddings are trained by the first block's goodness). The label overlay follows the reference implementation of Hinton [2022]: the first 10 entries of the flattened input are zeroed and the entry of the label is set to the maximum input value (for CIFAR-10 these are the first 10 pixels of the first row of channel 0); a wrong label is drawn uniformly from the nine other classes; the neutral label sets all 10 entries to 0.1 times the maximum. For PTB, 30 % of the positions of each sequence are replaced by uniformly random characters, resampled every step. The readout is trained every step by cross-entropy on the normalised, detached top-layer activity (neutral-label pass for images; the real sequences at every position for text). Goodness threshold $\theta = 2$ for all blocks except the LSTM block ($\theta = 0.25$, Section 3).

Evaluation. Validation and test metrics are computed with the plain forward of the full validation / test set at initialisation, every 5 minutes of training time (the clock is paused during evaluation) and at 60 minutes. For FF, evaluation uses the neutral-label pass and the trained readout.

Hyper-parameters. Table 7 summarises every hyper-parameter; none is tuned per method beyond the ε sweep.

Table 7: Hyper-parameters shared by all runs.

Optimizer	Adam, $(\beta_1, \beta_2) = (0.9, 0.99)$, $\epsilon_{\text{Adam}} = 10^{-8}$, no schedule, no weight decay
Learning rate = probe scale ε	$\{0.1, 0.01, 0.001\}$
Probe distribution	$\mathcal{N}(\mathbf{0}, \mathbf{I})$, scaled by ε
Forward passes per step	16 (probe methods), 2 (FF), 1 + 1 backward (BP)
SPSA antithetic pairs / RS candidates	8 / 15
Batch size	128 images; 64 sequences $\times$ 64 characters
Initialisation	Xavier-uniform, zero biases (LSTM forget gate = 1)
FF threshold θ / PTB corruption	2 (0.25 for the LSTM block) / 30 % of characters
Decorrelation rate η_R (DANP)	10^{-3}, fixed
Training budget	60 min wall-clock per run, evaluation excluded
Evaluation interval	every 5 min of training time, and at the end
Divergence rule	NaN, or training loss $> 2\times$ its initial value, or validation loss $> 2\times$ its step-0 value, at two consecutive evaluations
Seeds	1 per configuration

Data and licences. MNIST [LeCun et al., 1998], CIFAR-10 [Krizhevsky, 2009] and the character-level Penn Treebank files [Marcus et al., 1993, Mikolov et al., 2012] are used as distributed by their authors. MNIST is distributed under the Creative Commons Attribution-Share Alike 3.0 licence; CIFAR-10 is released by its authors for research use without a formal licence; the character-level PTB files are the widely redistributed Mikolov preprocessing of the Penn Treebank (LDC99T42) and are used for research only. Our code is released under the MIT licence.

C Per-task training curves for all ε

Figure 3 shows the test-loss curves of all 96 runs, one row per task and one column per ε.

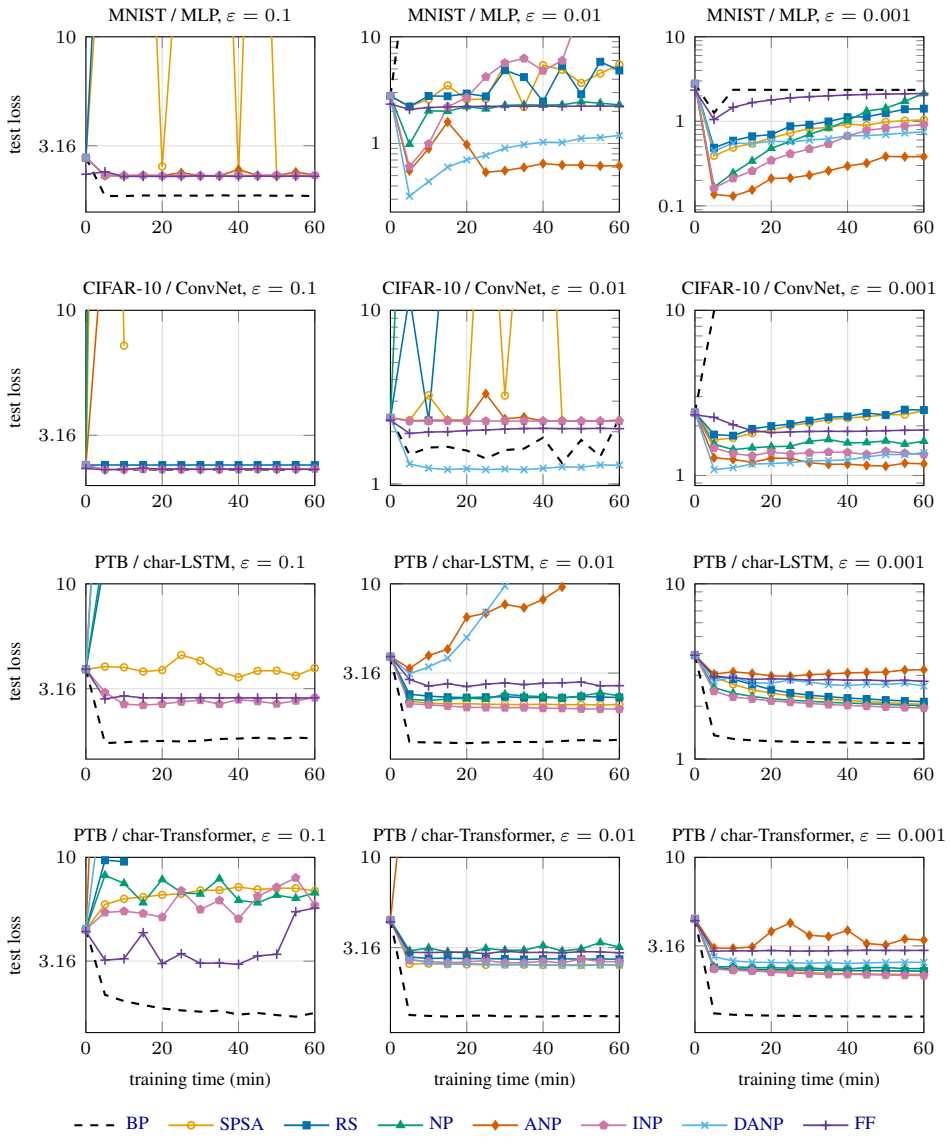


Figure 3: Test loss versus training time for every run. Rows: tasks; columns: $\varepsilon \in \{0.1, 0.01, 0.001\}$. Diverged runs end at their last finite evaluation.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 claim an empirical comparison of eight training rules on four small architectures under one protocol; this is exactly what Sections 3–5 deliver, and the scope (single seed, one hour, ≈40k parameters) is stated in Section 6.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 discusses the single seed, the one-hour budget, the 40k-parameter scale, the tying of ε to the learning rate, the dependence of wall-clock results on implementation efficiency, the status of FF and BP as non-probe references, and the character-level PTB choice.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper is purely empirical and contains no theoretical results; the estimator equations in Section 3.2 restate published rules with citations.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3 and 4 and Appendix B specify the architectures, data splits, estimators, budget, optimizer, sweep, selection rule and evaluation protocol; the code and all run logs are public.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The complete code, the run configurations, the per-run JSON results and the scripts that generate every table and figure are released at `https://github.com/Fchaubard/zero-order-ablations`; the datasets are public and downloaded by the code.

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 gives the data splits, batch sizes, initialisation, optimizer, the ε sweep and the validation-based selection rule; Appendix B (Table 7) lists every hyper-parameter.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Every configuration is run with a single seed because the full grid already requires 96 GPU-hours; Section 6 states this explicitly and warns that small differences should not be read as rankings.

   Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 4 states that every run uses one RTX 5090 for one hour of training (plus evaluation), 96 runs in total executed 8 at a time (10.5 hours of wall-clock on the 8-GPU node, about 100 GPU-hours including evaluation); the 32 two-minute smoke runs and the four supplementary 10-minute runs add about 2 GPU-hours, and the gradient-alignment measurements ran on a laptop CPU.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The study trains small models on three standard public benchmarks; it involves no human subjects, no personal data and no deployment.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: This is foundational optimizer-benchmarking research on ≈40k-parameter models with no direct application or deployment path; its potential positive impact (more energy-efficient or hardware-friendly training) is discussed in Section 1 and we see no specific negative impact beyond those of generic training algorithms.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: No models with misuse potential are released; the released artefacts are training code and logs of ≈40k-parameter models on MNIST, CIFAR-10 and character-level PTB.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: MNIST, CIFAR-10 and Penn Treebank are cited (Section 4) and their terms of use are stated in Appendix B; PyTorch is used under its BSD licence.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released code repository (MIT licence) documents the protocol (`PROTOCOL.md`), the command to launch every run and the scripts that regenerate every table and figure; the 96 result files and the supplementary runs are included.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper involves neither crowdsourcing nor human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: The paper involves no human subjects, so no IRB review was required.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [NA]

Justification: LLM-based assistants were used only for writing and coding assistance; no LLM is part of the methods, the experiments or the analysis.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (https://neurips.cc/Conferences/2025/LLM) for what should or should not be described.